

PEARLS AQI PREDICTOR

End-to-End Air Quality Index Forecasting System

Islamabad Forecast Lab

FINAL PROJECT REPORT

Prepared by: Mubashir Ahmed

BSc Software Engineering | UET Taxila

Project Focus: Data Science, Machine Learning, MLOps & Full-Stack Deployment

Reporting Date: August 31, 2026

1. Executive Summary

Pearls AQI Predictor is an end-to-end cloud-backed machine learning system designed to forecast the Air Quality Index (AQI) for Islamabad for the next three days. The project was developed as a complete product rather than as a standalone machine-learning notebook. It integrates external air-quality and weather data, feature engineering, cloud feature storage, automated model training, model evaluation, model registry, prediction serving, data-quality auditing, CI/CD-style scheduled workflows, and a production web dashboard.

The system uses Open-Meteo as the external data source, MongoDB Atlas as the cloud data and model-evidence store, FastAPI as the backend API layer, Next.js as the frontend, Render for backend hosting, Vercel for frontend hosting, and GitHub Actions for scheduled automation. The architecture separates data ingestion, feature engineering, training, registry, prediction, API delivery, and presentation so that each part can be maintained independently.

The forecasting pipeline creates one-day, two-day, and three-day future AQI targets and evaluates multiple candidate regressors. The reported horizon champions are Ridge Regression for Day +1 and Random Forest for Day +2 and Day +3. The project deliberately reports limitations and model performance rather than presenting the forecast as perfect or guaranteed.

2. Problem Statement

Air pollution changes over time because pollutant concentrations are influenced by recent air-quality conditions, meteorological behavior, seasonal patterns, and other environmental factors. A dashboard that only displays current AQI is useful for observation, but it does not provide forward-looking information.

The project therefore addresses the following problem: how can historical and recently observed air-quality and environmental signals be transformed into a maintainable machine-learning pipeline that produces useful three-day AQI forecasts for Islamabad and exposes those forecasts through a reliable web application?

2.1 Objectives

- Collect air-quality and weather observations for Islamabad from an external API.
- Create a clean, time-aware feature dataset suitable for supervised learning.
- Store processed features and machine-learning evidence in a cloud database.
- Generate Day +1, Day +2, and Day +3 forecasting targets.
- Train and compare multiple regression models.
- Select and register the best-performing model separately for each horizon.
- Store model artifacts and prediction evidence for reproducibility.
- Automate feature and training workflows using GitHub Actions.
- Expose forecasts, metrics, registry information, pipeline health, and quality status through FastAPI.
- Present the system through a product-style Next.js dashboard.

3. Project Scope and Requirements

Requirement	Implementation / Status
Three-day AQI forecast	Completed — API provides Day +1, Day +2 and Day +3

	forecasts.
External data integration	Completed — Open-Meteo weather and air-quality APIs.
Feature engineering	Completed — time, lag, rolling and AQI-movement features.
Historical backfill	Completed — historical hourly records prepared for training.
Cloud feature store	Completed — MongoDB Atlas.
Multiple ML models	Completed — Ridge, Random Forest, Gradient Boosting and MLP.
Model evaluation	Completed — RMSE, MAE and R ² recorded.
Model registry	Completed — champion metadata and model artifacts stored.
Automation	Completed — scheduled and manual GitHub Actions workflows.
Data-quality auditing	Completed — duplicate, null/range and leakage-oriented checks.
REST API	Completed — FastAPI endpoints for forecasts and evidence.
Web application	Completed — Next.js frontend deployed on Vercel.

4. System Architecture

The system follows a layered pipeline architecture. External data enters through the ingestion layer, is transformed into structured hourly features, and is persisted in MongoDB Atlas. The training layer creates future targets and evaluates candidate models. The registry records the selected champions and their metadata, while GridFS stores model binaries. The prediction service retrieves the latest registered model information and produces three-day forecasts. FastAPI exposes the results to the Next.js frontend.

Layer	Responsibility	Main Technology
Data source	Weather and pollutant observations	Open-Meteo APIs
Ingestion	Fetch, normalize and upsert recent observations	Python
Feature engineering	Lag, rolling, time and AQI-movement features	Python / pandas
Feature store	Persist processed hourly feature records	MongoDB Atlas
Training	Build targets, train candidates, evaluate	scikit-learn
Model registry	Champion metadata, metrics and feature list	MongoDB + GridFS
Prediction	Generate and persist three-day forecasts	Python / FastAPI
Backend	REST API and health/evidence endpoints	FastAPI
Frontend	Dashboard and project presentation	Next.js / React
Automation	Scheduled feature and training jobs	GitHub Actions
Deployment	Cloud hosting	Render + Vercel

5. Data Acquisition and Data Management

The project is configured for Islamabad, Pakistan, using coordinates 33.6844° N, 73.0479° E and the Asia/Karachi timezone. The ingestion process retrieves recent air-quality and environmental observations and converts them into consistent hourly records.

5.1 Data Sources

- Open-Meteo weather API — environmental/weather signals used by the feature pipeline.

- Open-Meteo air-quality API — pollutant observations used for AQI-related modeling.
- MongoDB Atlas — cloud persistence for features, predictions, metrics, registry records, quality audits and pipeline evidence.

5.2 Cloud Database Collections

Collection / Store	Purpose
features_v1	Processed Islamabad AQI and pollutant feature rows.
model_metrics	Evaluation metrics and model comparison results.
model_registry	Champion models, feature list, metrics, metadata and GridFS references.
predictions	Generated three-day forecasts and risk classifications.
pipeline_runs	Feature, training and audit execution evidence.
quality_audits	Quality checks including duplicates, AQI range and leakage-risk indicators.
fs.files / fs.chunks	GridFS storage for model artifacts.

The system is designed so credentials are not part of the application source. Database connection information is supplied through environment variables and deployment/automation secrets.

6. Feature Engineering

Raw observations are not passed directly to the forecasting models. The feature pipeline converts them into structured time-series predictors. This allows the models to use recent AQI movement, pollutant history and temporal context.

Feature Group	Examples	Purpose
Pollutants	pm10, pm2_5, co, no2, so2, o3	Represent current environmental conditions.
Time	day, month	Represent temporal/seasonal context.
AQI movement	aqi_delta	Represent recent change in AQI.
Lag features	aqi_lag_1, aqi_lag_2, aqi_lag_3	Capture recent historical behavior.
Pollutant lags	pm2_5_lag_1, pm10_lag_1	Capture short-term pollutant history.
Rolling features	pm2_5_roll_3, pm10_roll_3	Smooth short-term variation.

The latest project registry records a 17-column feature list comprising pollutant values, temporal variables, lag features, rolling features and AQI movement information.

[FIGURE 3 — Feature-engineering flow from raw hourly observations to model-ready records]

7. Machine Learning and Model Training

The training pipeline reads historical feature records from MongoDB Atlas and constructs future targets for each forecasting horizon. Separate targets are created for Day +1, Day +2 and Day +3. Candidate models are trained and evaluated, and the best model for each horizon is registered as that horizon's champion.

Model	Reason for Inclusion
Ridge Regression	Fast and interpretable linear baseline with regularization.
Random Forest	Captures non-linear relationships and interactions in tabular features.
Gradient Boosting	Strong tree-based challenger for structured data.
MLP Regressor	Neural-network-style challenger for non-linear tabular patterns.

7.1 Reported Model Results

Forecast Horizon	Champion	RMSE	MAE	R ²
Day +1	Ridge	12.19	8.88	0.507

Day +2	Random Forest	22.24	16.54	-0.673
Day +3	Random Forest	24.10	17.45	-0.973
Overall leaderboard	Random Forest	20.58 avg.	15.10 avg.	-0.446 avg.

These results show why horizon-level model selection is used. Forecast difficulty increases with horizon, and the same algorithm does not necessarily provide the strongest result for every future day. The negative R^2 values for Day +2 and Day +3 also demonstrate an important limitation: the current model configuration is not uniformly strong at longer horizons.

7.2 Evaluation Metrics

- MAE (Mean Absolute Error) measures the average absolute difference between predicted and actual AQI.
- RMSE (Root Mean Squared Error) gives greater weight to larger prediction errors.
- R^2 indicates how much variance is explained relative to a simple mean-based reference; negative values indicate weak predictive performance against that reference.

8. Model Registry and Artifact Management

The project treats trained models as deployable artifacts rather than temporary notebook files. The model registry records the champion selected for each forecast horizon, the feature list, evaluation metrics, leaderboard information and a reference to the stored model binary.

- Champion selection is performed per forecasting horizon.
- Model metadata is persisted in MongoDB Atlas.
- Model binaries are stored using GridFS.
- Prediction records retain the model used and forecast horizon.
- The dashboard/API can expose model evidence instead of hiding model selection logic.

9. Forecasting and Risk Classification

The backend provides a three-day forecast. In horizon-champion mode, each day can use the model selected for that horizon. This means the system can use Ridge for Day +1 while using Random Forest for Day +2 and Day +3.

Horizon	Example Forecast in Source Evidence	Risk
Day +1	87.89	Moderate
Day +2	86.41	Moderate
Day +3	100.50	Unhealthy for Sensitive Groups

The API also supports model override behavior so that the dashboard can compare predictions generated using a selected model rather than only the automatically selected champion.

10. MLOps and GitHub Actions Automation

Automation is a major part of the system. The project uses GitHub Actions to reduce dependence on manual terminal execution. The feature workflow runs on an hourly schedule and can also be triggered manually. The training workflow runs on a daily schedule with catch-up behavior and manual recovery support.

Workflow	Schedule / Trigger	Main Actions
----------	--------------------	--------------

Feature Pipeline	Hourly + backup schedule + manual	Fetch recent data, engineer features, deduplicate and write to Atlas.
Training Pipeline	Daily + backup/catch-up + manual	Train models, evaluate, update registry, store artifacts and predictions.
Recovery	Manual	Allow controlled reruns when scheduled execution is delayed or needs recovery.

Backup schedules were introduced because scheduled runners can be delayed. Deduplication and catch-up logic make repeated or delayed execution safer and reduce the risk of corrupting the feature store.

11. Data Quality, EDA and Explainability

Forecasting systems can produce plausible-looking outputs even when the underlying dataset contains duplicates, missing values, invalid ranges or leakage. The project therefore includes explicit quality auditing.

Quality Check	Latest Reported Result
Rows audited	2208
Duplicate rows	0
AQI out of range	0
Leakage risk hint	False
Overall status	PASS

The project also produces exploratory analysis artifacts, including summary statistics, correlation analysis, daily AQI trend visualizations and feature-importance-style evidence. These artifacts provide a bridge between raw data, model behavior and the final dashboard.

12. Backend API

FastAPI provides the application interface between the machine-learning/data layer and the frontend. The API is designed around clear responsibilities: health, predictions, model evidence, metrics, pipeline status and data quality.

Endpoint	Purpose
/health	Check service and database health.
/predict	Return the latest three-day AQI forecast and risk levels.
/metrics/latest	Return latest model metrics and leaderboard information.
/models/latest	Return latest registry metadata and champion models.
/pipeline/health	Return recent pipeline execution evidence.
/quality/latest	Return latest data-quality audit.

The deployed backend is hosted on Render. Interactive API documentation is provided by FastAPI's generated documentation interface.

13. Frontend and Product Design

The frontend is implemented with Next.js and React. Rather than exposing raw API responses, it turns the forecasting pipeline into a product-style interface. The dashboard communicates with the FastAPI backend and presents forecast values, risk levels, model comparison, registry evidence, quality checks and pipeline health.

- Landing page introduces the project and its purpose.

- Dashboard presents the operational forecasting experience.
- Methodology content explains the data and ML pipeline.
- The frontend is connected to the deployed backend through environment configuration.
- The application is deployed on Vercel.

14. Deployment and Cloud Infrastructure

Component	Platform / Service	Role
Frontend	Vercel	Host and deploy Next.js application.
Backend	Render	Host FastAPI API service.
Database	MongoDB Atlas	Cloud feature store, registry, metrics, predictions and audit evidence.
Model artifacts	MongoDB GridFS	Persist trained model binaries.
Automation	GitHub Actions	Scheduled feature and training workflows.

The production architecture therefore separates presentation, application/API, persistent data, model artifacts and automation. This separation makes the project closer to a deployable software system than a local ML experiment.

15. Security and Configuration

- MongoDB credentials and connection information are kept outside committed source code.
- GitHub Actions receives sensitive configuration through repository secrets.
- Render receives sensitive configuration through environment variables.
- The frontend communicates with public API endpoints and does not contain database credentials.
- Public project evidence avoids exposing secret values.

Environment-based configuration also makes it possible to use local development settings separately from cloud deployment settings.

16. Challenges and Engineering Solutions

Challenge	Solution / Engineering Response
GitHub account / billing restrictions	Moved workflow execution to a clean repository configuration so Actions could operate normally.
MongoDB connection failure	Corrected connection configuration, including URL-safe credential handling, and kept the URI outside source code.
Incorrect historical city data	Reconfigured ingestion for Islamabad and reran the required ingestion/training stages.
Scheduled workflows appeared delayed	Added backup cron schedules and training catch-up logic.
Frontend could not initially reach backend	Connected the frontend environment to the Render API and configured CORS.
Different model behavior by horizon	Implemented horizon-level champion selection instead of forcing one model across all three days.
Need for operational proof	Added registry, metrics, quality audits, pipeline logs and deployment evidence.

These challenges demonstrate that the project involved software engineering and operational problem-solving in addition to machine-learning experimentation.

17. Limitations

A strong technical report should distinguish between what the system can do and what the current evidence proves. The current results show useful Day +1 performance but substantially weaker Day +2 and Day +3 performance. The reported negative R^2 values for the longer horizons indicate that the current feature/model configuration should not be interpreted as a highly accurate long-range predictor.

- The system currently focuses on Islamabad rather than multiple cities.
- Longer-horizon model performance needs further improvement and validation.
- Forecast quality depends on the availability and quality of upstream API observations.
- The feature set is engineered from available pollutant, temporal and recent-history signals and can be expanded.
- Model performance can change as new observations enter the system; production monitoring and periodic retraining remain important.

These limitations do not invalidate the engineering value of the project. They identify clear next steps for improving the forecasting component while retaining the existing MLOps and deployment architecture.

18. Future Enhancements

- Expand the system from Islamabad to additional Pakistani cities.
- Add richer meteorological features such as wind direction/speed, temperature, humidity and pressure where available.
- Experiment with stronger time-series methods and carefully tuned gradient-boosting models.
- Perform systematic hyperparameter optimization and rolling-origin validation.
- Introduce model drift and forecast-error monitoring.
- Add historical forecast-vs-actual evaluation to the dashboard.
- Improve explainability with stronger feature-attribution analysis.
- Add alerting for unusually high predicted AQI.
- Introduce versioned datasets and model releases for stronger reproducibility.

19. Code Organization

Project Area	Responsibility
backend/app	FastAPI application, database access, feature engineering, training and prediction logic.
backend/scripts	Command-line entry points used by automated workflows.
frontend/app	Next.js pages, dashboard and UI components.
.github/workflows	Feature, training and recovery automation.
documentation	Reports, screenshots, evidence and supporting documentation.
backend/artifacts	EDA and local/model evidence artifacts.

The separation of backend, frontend, automation, documentation and artifacts makes the repository easier to navigate and maintain.

20. Deployment and Submission References

Resource	Reference
GitHub repository	https://github.com/mubashirahmed0296-pixel/aqi-predictor
Frontend	Vercel deployment configured for the Pearls AQI Predictor

	frontend
Backend	https://aqi-predictor-off9.onrender.com
API documentation	https://aqi-predictor-off9.onrender.com/docs
Database	MongoDB Atlas — aqi_predictor database

Note: deployment URLs can change when services are recreated or promoted between preview and production deployments. The repository should be treated as the primary source of current deployment references.

21. Verification and Evidence

The system was verified through multiple layers rather than through a single successful page load. Evidence includes database records, model metrics, registry entries, prediction records, quality audits, API documentation, deployment status and automated workflow history.

Verification Area	Evidence
Data pipeline	Feature records and pipeline run records in Atlas.
Modeling	Stored model metrics and champion registry records.
Artifacts	GridFS model references and stored binaries.
Prediction	Three-day prediction records and API response.
Quality	Latest audit reports PASS with zero duplicates and zero AQI out-of-range records in source evidence.
Backend	FastAPI health and documentation endpoints.
Automation	GitHub Actions scheduled/manual workflow history.
Frontend	Deployed Next.js application connected to backend.

22. Conclusion

Pearls AQI Predictor demonstrates how a machine-learning model can be transformed into an end-to-end software product. The core contribution is not simply predicting AQI; it is integrating the complete lifecycle around the prediction: data acquisition, feature engineering, cloud storage, model comparison, horizon-level model selection, model artifact management, automated execution, quality auditing, API delivery, and a user-facing dashboard.

The current system provides a working three-day Islamabad AQI forecasting workflow with cloud infrastructure and operational evidence. Its model results are reported honestly: Day +1 shows the strongest performance, while longer horizons require further research and improvement. This makes the project both a functional application and a strong foundation for continued work in machine learning, MLOps, data engineering and full-stack AI systems.

Overall, the project demonstrates practical skills across Python, machine learning, FastAPI, Next.js, MongoDB Atlas, GridFS, GitHub Actions, Render and Vercel, while maintaining a clear separation between experimentation, production services and operational evidence.

Appendix A — Technology Stack

Category	Technology
Programming	Python, TypeScript / JavaScript
ML	scikit-learn, pandas
Backend	FastAPI
Frontend	Next.js 14.2.13, React 18.3.1
Database	MongoDB Atlas
Model storage	MongoDB GridFS
Automation	GitHub Actions
Backend hosting	Render
Frontend hosting	Vercel
Data provider	Open-Meteo

Appendix B — Key Project Configuration

Parameter	Value
Project	Pearls AQI Predictor — Islamabad Forecast Lab
City	Islamabad, Pakistan
Latitude	33.6844
Longitude	73.0479
Timezone	Asia/Karachi
Forecast horizon	3 days
Primary database	aqi_predictor